```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

# Set random seed for reproducibility
np.random.seed(42)

# Set plotting style
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
plt.style.available else 'default')

def true_function(X):
    """The underlying ground truth function."""
    return np.sin(1.5 * np.pi * X)

# Generate 100 sample data points
n_samples = 100
X = np.sort(np.random.rand(n_samples))
y = true_function(X) + np.random.normal(0, 0.2, size=n_samples)

# Split into Training (60%) and Testing (40%) sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=42
)

# Reshape X for Scikit-Learn (requires 2D array)
X_train_vec = X_train[:, np.newaxis]
X_test_vec = X_test[:, np.newaxis]

# Smooth X values for plotting continuous model prediction curves
X_plot = np.linspace(0, 1, 200)[:, np.newaxis]

print(f"Training samples: {len(X_train)} | Testing samples: {len(X_test)}")

Training samples: 60 | Testing samples: 40

degrees = [1, 3, 15]

plt.figure(figsize=(16, 5))

for i, degree in enumerate(degrees):
    ax = plt.subplot(1, len(degrees), i + 1)

    # Build Polynomial Regression pipeline
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
```

```
    model.fit(X_train_vec, y_train)

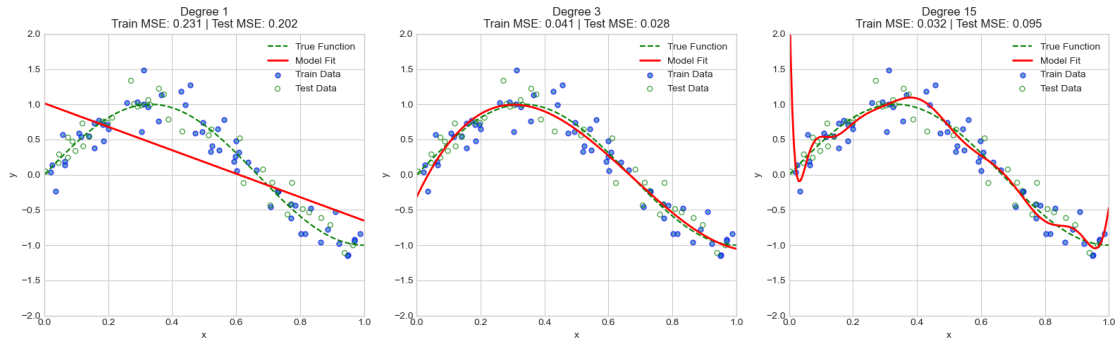
    # Evaluate predictions
    y_plot_pred = model.predict(X_plot)
    y_train_pred = model.predict(X_train_vec)
    y_test_pred = model.predict(X_test_vec)

    # Calculate Mean Squared Error (MSE)
    train_mse = mean_squared_error(y_train, y_train_pred)
    test_mse = mean_squared_error(y_test, y_test_pred)

    # Plot True Function vs Model Predictions
    plt.plot(X_plot, true_function(X_plot), label="True Function",
color="green", linestyle="--")
    plt.plot(X_plot, y_plot_pred, label="Model Fit", color="red",
linewidth=2)
    plt.scatter(X_train, y_train, edgecolor='b', s=25, label="Train Data",
alpha=0.7)
    plt.scatter(X_test, y_test, edgecolor='g', facecolors='none', s=25,
label="Test Data", alpha=0.7)

    plt.xlabel("x")
    plt.ylabel("y")
    plt.xlim((0, 1))
    plt.ylim((-2, 2))
    plt.legend(loc="best")
    plt.title(f"Degree {degree}\nTrain MSE: {train_mse:.3f} | Test MSE:
{test_mse:.3f}")

plt.tight_layout()
plt.show()
```




```
max_degree = 12
degrees_range = range(1, max_degree + 1)

train_errors = []
test_errors = []

for degree in degrees_range:
```

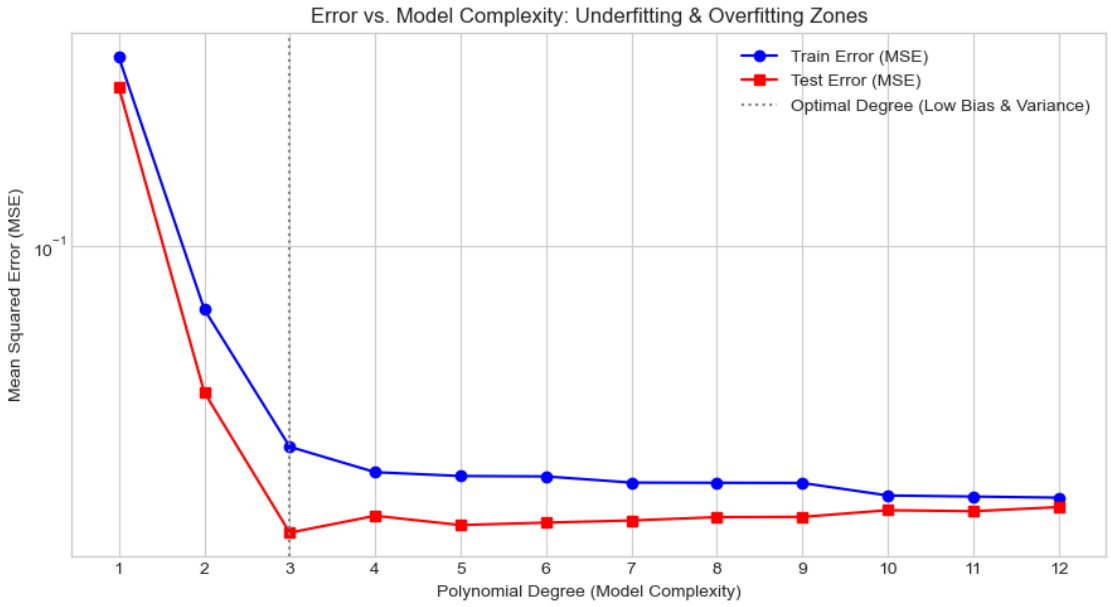
```
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
    model.fit(X_train_vec, y_train)

    train_mse = mean_squared_error(y_train, model.predict(X_train_vec))
    test_mse = mean_squared_error(y_test, model.predict(X_test_vec))

    train_errors.append(train_mse)
    test_errors.append(test_mse)

# Plotting the Train/Test Error curves
plt.figure(figsize=(9, 5))
plt.plot(degrees_range, train_errors, label="Train Error (MSE)", marker='o',
color='blue')
plt.plot(degrees_range, test_errors, label="Test Error (MSE)", marker='s',
color='red')

plt.xlabel("Polynomial Degree (Model Complexity)")
plt.ylabel("Mean Squared Error (MSE)")
plt.yscale('log')  # Log scale used to handle large test MSE values during
extreme overfitting
plt.xticks(degrees_range)
plt.title("Error vs. Model Complexity: Underfitting & Overfitting Zones")
plt.axvline(x=3, color='gray', linestyle=':', label="Optimal Degree (Low Bias
& Variance)")
plt.legend()
plt.tight_layout()
plt.show()
```




```
n_bootstraps = 100
degrees = range(1, 11)
eval_points = np.linspace(0, 1, 100)[:, np.newaxis]
```

```
# Store results
biases_sq = []
variances = []
total_errors = []

for degree in degrees:
    predictions = np.empty((n_bootstraps, len(eval_points)))

    for b in range(n_bootstraps):
        # Resample training data with replacement
        indices = np.random.choice(len(X_train), size=len(X_train),
replace=True)
        X_boot = X_train_vec[indices]
        y_boot = y_train[indices]

        # Fit model on bootstrap sample
        model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
        model.fit(X_boot, y_boot)
        predictions[b, :] = model.predict(eval_points)

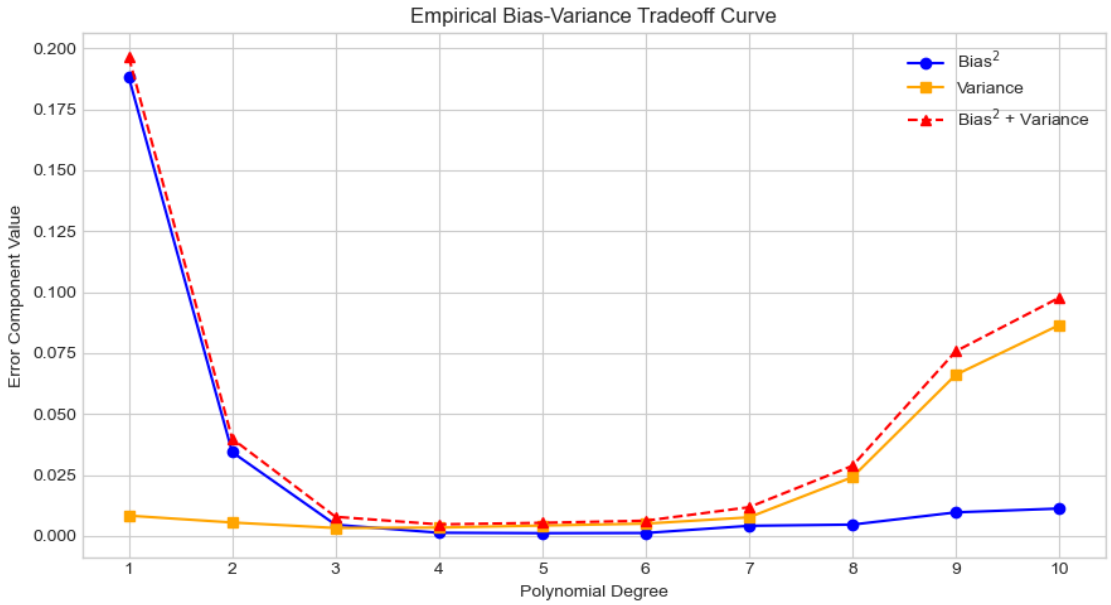
    # Compute ground truth at evaluation points
    y_true_eval = true_function(eval_points).ravel()

    # Calculate Bias^2 and Variance across points
    pred_mean = np.mean(predictions, axis=0)
    bias_sq = np.mean((y_true_eval - pred_mean) ** 2)
    variance = np.mean(np.var(predictions, axis=0))

    biases_sq.append(bias_sq)
    variances.append(variance)
    total_errors.append(bias_sq + variance)

# Plotting Bias-Variance Tradeoff
plt.figure(figsize=(9, 5))
plt.plot(degrees, biases_sq, label=r"Bias$^2$", color='blue', marker='o')
plt.plot(degrees, variances, label="Variance", color='orange', marker='s')
plt.plot(degrees, total_errors, label=r"Bias$^2$ + Variance", color='red',
linestyle='--', marker='^')

plt.xlabel("Polynomial Degree")
plt.ylabel("Error Component Value")
plt.title("Empirical Bias-Variance Tradeoff Curve")
plt.xticks(degrees)
plt.legend()
plt.tight_layout()
plt.show()
```


Empirical Bias-Variance Tradeoff Curve
Bias^2
Variance
Bias^2 + Variance
Error Component Value
Polynomial Degree
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
1
2
3
4
5
6
7
8
9
10